

Department of Computer Science and Engineering

Global Campus, Jakkasandra Post, Kanakapura Taluk, Ramanagara District, Pin Code: 562 112

2023-2027

Sixth Semester Progress Report on

“Website Phishing Tracker”

Submitted for the partial fulfillment of Project Centric Learning activity of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

**Akhilesh Reddy
23BTRCT007**

**Amarthya K
23BTRCT009**

**Athul Raj
23BTRCT048**

**Divaakar S
23BTRCT049**

**Under the guidance of
Dr. S. Jerald Nirmal Kumar,**
Assistant Professor
Department of Computer Science &
Engineering

School of Computer Science and Engineering
JAIN (Deemed-to-be University)

Department of Computer Science and Engineering

Global Campus, Jakkasandra Post, Kanakapura Taluk, Ramanagara District, Pin Code: 562 112

CERTIFICATE

This is to certify that the project centric learning work titled “**Website Phishing Tracker**” is carried out by Akhilesh Reddy(23BTRCT007), Amarthya K(23BTRCT009), Athul Raj(23BTRCT048), Divaakar S(23BTRCT049), bonafide students of Bachelor of Technology at the Faculty of Engineering & Technology, Jain Deemed-to-be University, Bangalore in partial fulfillment for the project centric learning activity of degree in Bachelor of Technology in Computer Science & Engineering, during the year **2024-2025**.

<i>Dr. S. Jerald Nirmal Kumar</i>	<i>Dr. Mahesh T.R</i>
Assistant Professor, Dept. of CSE, School of Computer Science and Engineering JAIN (Deemed-to-be University)	<i>Program Head,</i> Dept. of CSE, School of Computer Science and Engineering JAIN(Deemed-to-be University)

Name of the Examiner

Signature of Examiner

1.

DECLARATION

We, **Akhilesh Reddy(23BTRCT007), Amarthya K(23BTRCT009), Athul Raj (23BTRCT048), Divaakar S(23BTRCT049)**, are student's of 2nd semester B.Tech in **Computer Science & Engineering**, JAIN(Deemed-to-be University), hereby declare that the project centric learning work titled “Fake Website detecting and Phishing” has been carried out by us and submitted in partial fulfilment for the project centric learning activity of degree in **Bachelor of Technology in Computer Science & Engineering** during the academic year **2023-2027**.

Signature

Name1: Akhilesh Reddy USN: 23BTRCT007	
Name 2: Amarthya K USN: 23BTRCT009	
Name 3: Athul Raj USN: 23BTRCT048	
Name 4: Divaakar S USN: 23BTRCT049	

Place : Bangalore

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of School of Computer Science and Engineering, Jain (Deemed-to-be University) for the progress of this project centric learning work.

*In particular we would like to thank **Dr.Geetha, Director**, School of Computer Science Engineering and **Prof.Mahesh T.R, Program Head**, Department of Computer Science and Engineering, JAIN (Deemed-to-be University) for their constant encouragement and expert advice.*

*We would like to thank our guide **Dr.S. Jerald Nirmal Kumar, Designation, Department of Computer Science & Engineering**, JAIN (Deemed-to-be University), for sparing his valuable time to extend help in every step of our project work, which paved the way for smooth progress of the project.*

*We would like to thank our Project Coordinators **Dr.S. Jerald Nirmal Kumar** and all the staff members of Computer Science & Engineering for their support.*

We would like to thank one and all who directly or indirectly helped us in the progress of our Project.

Signature of Students

ABSTRACT

Phishing is one of the most prevalent cybersecurity threats, where attackers deceive individuals into revealing sensitive information such as passwords, banking credentials, and personal data. With the increasing sophistication of phishing techniques, there is a growing need for effective awareness and detection systems. This project aims to develop a Phishing Awareness System using Python, focusing on backend functionalities that analyze phishing threats, educate users, and provide real-time detection mechanisms.

The backend system will utilize Python-based security frameworks and threat analysis techniques to detect phishing attempts. It will integrate phishing link analysis, a threat intelligence database, and automated reporting mechanisms to help users identify potential threats. Additionally, the system will offer educational resources, phishing simulations, and awareness modules to enhance user knowledge about phishing attacks. By leveraging Flask or Django, along with machine learning algorithms for phishing detection, the project aims to create a scalable and efficient cybersecurity solution.

This system will not only provide real-time phishing detection but also serve as a knowledge hub for cybersecurity awareness. Future enhancements may include AI-powered threat detection, real-time phishing email analysis, and API integrations for broader cybersecurity applications. By implementing this project, we aim to minimize phishing-related risks and promote a safer online environment.

TABLE OF CONTENTS

	Page No
Abstract	v
Chapter 1	01-04
1. INTRODUCTION	
Overview	01
Background and Motivation	02
Chapter 2	05-08
2. Literature Survey	05
Chapter 3	09-14
3. Objective and Methodology	
3.1 Objective	09
3.2 Methodology	09
3.3 Algorithm Used	12
Chapter 4	15-17
4. System Design	
4.1 System Architecture	15
4.2 Data Flow diagram	15
4.3 Implementation	16
Chapter 5	18
5. Hardware and Software requirement	
5.1 Hardware requirement	18
5.2 Software requirement	18
Chapter 6	19
6. Result	19
Conclusion	20
References	20
Appendix – 1	22

Chapter 1

INTRODUCTION

INTRODUCTION

Overview

Phishing attacks have become one of the most dangerous cybersecurity threats affecting individuals, organizations, and online platforms worldwide. Attackers use deceptive techniques such as fake websites, malicious URLs, cloned login pages, and fraudulent messages to steal confidential information including usernames, passwords, banking credentials, and personal details. According to recent cybersecurity reports, phishing attacks increased significantly in 2024, with millions of malicious URLs being detected globally. The rapid growth of AI-generated phishing campaigns and visually deceptive websites has made traditional blacklist-based detection methods insufficient.

This project introduces PhishGuard, a Real-Time Phishing URL Detection System developed using machine learning and deep learning techniques. The proposed system focuses on detecting phishing websites in real time through the analysis of lexical URL patterns, domain-based characteristics, and lightweight webpage content indicators. The system utilizes a hybrid ensemble architecture integrating Random Forest, XGBoost, and Character-Level Convolutional Neural Networks (CNN) for accurate phishing classification.

The proposed system is designed as a browser extension capable of instantly analyzing URLs whenever a user visits a webpage. The extension displays visual indicators showing whether a website is safe, suspicious, or phishing. Unlike

conventional blacklist systems, PhishGuard can identify zero-day phishing attacks using intelligent feature-based analysis and neural pattern recognition.

The main objective of this project is to develop a fast, accurate, and lightweight phishing detection framework capable of operating in real-time environments while preserving user privacy. The system minimizes dependence on external APIs and performs most processing locally to ensure low latency and secure browsing.

By developing PhishGuard, this research contributes toward improving cybersecurity awareness, protecting users from phishing attacks, and enhancing real-time web security using ensemble machine learning approaches.

Background and Motivation

Background

Phishing is one of the most widespread forms of cybercrime and continues to evolve rapidly with advancements in web technologies and artificial intelligence. Attackers create fake websites that visually resemble legitimate platforms such as banking portals, social media websites, educational platforms, and e-commerce applications. These phishing websites trick users into entering confidential information which is later misused for identity theft, financial fraud, and unauthorized access.

Traditional phishing detection systems mainly rely on blacklists and rule-based heuristics. Although these methods can identify previously known phishing domains, they fail to detect newly generated phishing URLs and sophisticated attacks involving URL obfuscation, subdomain manipulation, and AI-generated phishing pages. Moreover, external blacklist APIs introduce additional latency and privacy concerns because user browsing data must be shared with third-party services.

Machine learning and deep learning techniques provide a more scalable and intelligent approach to phishing detection. By learning patterns from large datasets of phishing and legitimate URLs, machine learning algorithms can identify suspicious behavior even for previously unseen attacks. Ensemble learning models combining multiple classifiers have demonstrated strong performance in reducing false positives and improving detection accuracy.

Motivation

The increasing number of phishing attacks and the limitations of traditional security solutions motivated the development of PhishGuard. Existing systems either focus only on URL blacklisting or provide enterprise-level protection without lightweight browser-based solutions for normal users.

This project is motivated by the need to create a real-time phishing detection system that:

- Detects phishing URLs with high accuracy.

- Works efficiently within browser environments.

- Provides instant feedback to users.

- Maintains low processing latency.

- Preserves user privacy through local analysis.

- Handles zero-day phishing attacks.

The proposed PhishGuard system integrates lexical analysis, domain analysis, content inspection, and neural network-based pattern learning into a hybrid ensemble framework. The use of Random Forest and XGBoost improves structured feature analysis, while the Character-Level CNN identifies obfuscated and visually

deceptive phishing URLs.

The system also includes a browser extension prototype that visually alerts users using color-coded indicators:

Green – Safe website

Yellow – Suspicious website

Red – Phishing website

By combining real-time analysis with ensemble learning techniques, PhishGuard aims to provide a practical and deployable solution for modern phishing detection.

Chapter 2

LITERATURE SURVEY

1. Evolution of Phishing Attacks

Phishing attacks have evolved significantly over the years, becoming more sophisticated and difficult to detect. Early phishing attacks mainly involved fake emails and simple spoofed websites. Modern phishing attacks now use AI-generated emails, domain spoofing, URL obfuscation, cloned login pages, and social engineering techniques to deceive users.

Recent studies show that phishing attacks increasingly target banking systems, educational institutions, social media platforms, and cloud services. Attackers continuously modify URL structures and webpage layouts to bypass traditional blacklist systems. This rapid evolution highlights the importance of intelligent and adaptive phishing detection systems.

2. Existing Phishing Detection Techniques

Blacklist and Heuristic-Based Detection

Traditional phishing detection approaches rely heavily on blacklist databases and heuristic rules. Blacklist systems compare URLs against previously known phishing domains. Although blacklist methods provide low-latency detection, they fail against zero-day phishing attacks because newly generated phishing URLs are not present in existing databases.

Heuristic-based approaches analyze URL structures, suspicious keywords, HTTPS usage, special characters, and webpage characteristics. However, attackers can bypass static heuristic rules using obfuscation and dynamically generated phishing

pages.

Machine Learning-Based Detection

Machine learning algorithms have shown strong performance in phishing detection tasks. Various studies have applied Decision Trees, Support Vector Machines (SVM), Random Forest, Logistic Regression, and XGBoost for classifying URLs as phishing or legitimate.

Random Forest algorithms provide high classification accuracy by combining multiple decision trees and reducing overfitting. XGBoost improves phishing detection performance through gradient boosting and optimized feature learning.

Several studies also demonstrate that feature-based machine learning methods using lexical features, domain-based indicators, and webpage content characteristics significantly improve phishing detection accuracy.

Deep Learning Approaches

Deep learning techniques such as Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) are increasingly used for phishing detection.

Character-Level CNN models process raw URL strings directly without requiring extensive manual feature engineering. CNNs are capable of identifying hidden patterns, URL obfuscation techniques, and homographic attacks.

Research also shows that combining CNNs with attention mechanisms and ensemble learning strategies improves robustness against sophisticated phishing attacks.

3. Ensemble Learning for Phishing Detection

Ensemble learning combines multiple machine learning algorithms to improve overall prediction performance. Studies indicate that ensemble approaches outperform individual classifiers because they reduce variance and improve generalization.

Random Forest and XGBoost models perform well on structured URL and domain features, while CNN models effectively identify sequential URL patterns and hidden obfuscation techniques.

Hybrid phishing detection systems combining tree-based models and neural networks achieve:

- Higher accuracy

- Reduced false positives

- Better zero-day attack detection

- Improved robustness across datasets

These findings motivate the use of a hybrid ensemble architecture in PhishGuard.

4. Research Gap

Although existing phishing detection systems achieve good classification performance, several limitations remain:

- Many systems rely heavily on blacklist databases.

- Existing methods introduce high latency due to external API calls.

Some deep learning approaches are computationally expensive.
Many solutions are not optimized for browser-based deployment.
Existing systems often lack real-time responsiveness.

Most current phishing detection solutions are enterprise-focused and not designed for lightweight deployment on user devices.

5. Need for PhishGuard

The proposed PhishGuard system addresses these limitations by introducing:

Real-time phishing URL detection.
Hybrid ensemble learning architecture.
Browser extension deployment.
Lightweight feature extraction.
Local processing for privacy preservation.
Low-latency detection under 50 milliseconds.

The system combines lexical, domain-based, and content-based features with machine learning and deep learning models to provide robust phishing detection suitable for real-world deployment.

Chapter 3

OBJECTIVE AND METHODOLOGY

3.1 Objective

The primary objective of this project is to develop a real-time phishing URL detection system called PhishGuard using ensemble machine learning and deep learning techniques.

The objectives of the system are:

1. Detect phishing URLs using machine learning and deep learning models.
2. Extract lexical, domain-based, and content-based features from websites.
3. Implement a hybrid ensemble framework using Random Forest, XGBoost, and Character-Level CNN.
4. Provide real-time phishing detection through a browser extension.
5. Reduce false positives and false negatives.
6. Detect zero-day phishing attacks without relying completely on blacklists.
7. Ensure low latency and lightweight deployment.
8. Preserve user privacy through local URL analysis.

3.2 Methodology

The development methodology of PhishGuard follows multiple stages:

1. Data Collection

Phishing URLs are collected from sources such as:

- PhishTank
- OpenPhish
- Community phishing feeds

Legitimate URLs are collected from trusted domains and popular websites.

2. Data Preprocessing

The collected URLs undergo preprocessing steps including:

- Duplicate removal
- URL normalization
- Lowercase conversion
- Noise filtering
- Feature standardization

Balanced datasets are created using stratified sampling.

3. Feature Extraction

The system extracts multiple categories of features:

Lexical Features

- URL length
- Special character count
- Number of subdomains
- Presence of suspicious keywords

Domain Features

- Domain age
- WHOIS information
- DNS indicators

Content Features

- Form count
- iframe usage
- External resource ratio
- JavaScript behavior

4. Model Training

Three machine learning models are trained:

- Random Forest
- XGBoost
- Character-Level CNN

Tree-based models process structured feature vectors while CNN processes raw URL character sequences.

5. Ensemble Learning

The outputs from all models are combined using weighted voting:

$$P_{\text{final}} = w_1(\text{PRF}) + w_2(\text{PXGB}) + w_3(\text{PCNN})$$

The final score determines whether the URL is phishing or legitimate.

6. Browser Extension Integration

The trained model is integrated into a browser extension capable of:

- Capturing URLs in real time
- Running phishing analysis
- Displaying instant visual alerts
- Blocking suspicious websites

7. Performance Evaluation

The system is evaluated using:

- Accuracy
- Precision
- Recall
- F1-score

- ROC/AUC
- Runtime latency

The goal is to achieve high detection accuracy while maintaining low processing time.

3.3 Algorithm Used

Hybrid Ensemble Phishing Detection Algorithm (HEPDA)

The proposed PhishGuard system uses a Hybrid Ensemble Phishing Detection Algorithm (HEPDA) that combines Random Forest, XGBoost, and Character-Level CNN models.

The algorithm performs phishing detection in four stages:

1. Feature Extraction
2. Parallel Model Prediction
3. Ensemble Aggregation
4. Real-Time Decision Generation

The Random Forest model handles structured lexical and domain-based features. XGBoost improves optimized feature learning through gradient boosting techniques. The Character-Level CNN processes raw URL sequences to detect hidden phishing patterns and URL obfuscation.

The final phishing score is computed using weighted ensemble voting:

$$P_{\text{final}} = w_1(\text{PRF}) + w_2(\text{PXGB}) + w_3(\text{PCNN})$$

If the final score exceeds a predefined threshold, the URL is classified as phishing; otherwise, it is classified as legitimate.

The proposed hybrid ensemble architecture improves:

- Detection accuracy
- Robustness against zero-day attacks

- Detection of obfuscated URLs
- Real-time browser performance

This algorithm forms the core detection engine of PhishGuard and enables efficient phishing detection suitable for practical deployment environments.

```

phishing_tracker_backend.py >
51 def check_google_safebrowsing(url):
52     """Checks the URL against Google Safe Browsing API."""
53     try:
54         request_data = {
55             "client": {"clientId": "phishing_tracker", "clientVersion": "1.0"},
56             "threatInfo": {
57                 "threatTypes": ["MALWARE", "SOCIAL_ENGINEERING"],
58                 "platformTypes": ["ANY_PLATFORM"],
59                 "threatEntryTypes": ["URL"],
60                 "threatEntries": [{"url": url}],
61             },
62         }
63         response = requests.post(
64             f"https://safebrowsing.googleapis.com/v4/threatMatches:find?key={GOOGLE_API_KEY}",
65             json=request_data,
66         )
67         response.raise_for_status()
68         data = response.json()
69         return bool(data.get("matches"))
70     except requests.exceptions.RequestException as e:
71         logging.error(f"Google Safe Browsing API error: {e}")
72         return None
73
74 @app.route('/check')
75 def check():
76     """Endpoint to check if a URL is safe or phishing."""
77     url = request.args.get('url')
78     if not url or not is_valid_url(url):
79         return jsonify({"result": "Error: Invalid URL provided."}), 400
80
81     database = load_database()
82
83     if check_local_database(url, database):
84         return jsonify({"result": "Phishing (Local Database)"})
85
86     if check_openphish(url):
87         return jsonify({"result": "Phishing (OpenPhish)"})
88
89     if check_google_safebrowsing(url):
90         return jsonify({"result": "Phishing (Google Safe Browsing)"})
91
92     return jsonify({"result": "Safe"})
93
94 if __name__ == '__main__':
95     app.run(debug=True, port=5000)
96

```

```

35 popups > -
1 document.addEventListener("DOMContentLoaded", function () {
2     // Attach event listener to the button
3     document.getElementById("check-button").addEventListener("click", checkWebsite);
4 });
5
6 function checkWebsite() {
7     document.getElementById("result").textContent = "Checking...";
8     document.getElementById("result").style.visibility = "visible";
9
10    // Get the active tab's URL
11    chrome.tabs.query({ active: true, currentWindow: true }, function (tabs) {
12        if (tabs.length === 0) {
13            document.getElementById("result").textContent = "Error: No active tab found.";
14            document.getElementById("result").classList.add("error");
15            return;
16        }
17
18        let url = tabs[0].url;
19        fetch("http://127.0.0.1:5000/check?url=${encodeURIComponent(url)}")
20            .then(response => response.json())
21            .then(data => {
22                let resultDiv = document.getElementById("result");
23                resultDiv.textContent = data.result;
24                resultDiv.classList.remove("safe", "phishing", "error");
25
26                if (data.result.includes("Phishing")) {
27                    resultDiv.classList.add("phishing");
28                } else if (data.result === "Safe") {
29                    resultDiv.classList.add("safe");
30                } else {
31                    resultDiv.classList.add("error");
32                }
33            })
34            .catch(error => {
35                console.error("Error:", error);
36                let resultDiv = document.getElementById("result");
37                resultDiv.textContent = "Error checking the website.";
38                resultDiv.classList.add("error");
39            });
40    });
41 }
42

```

```

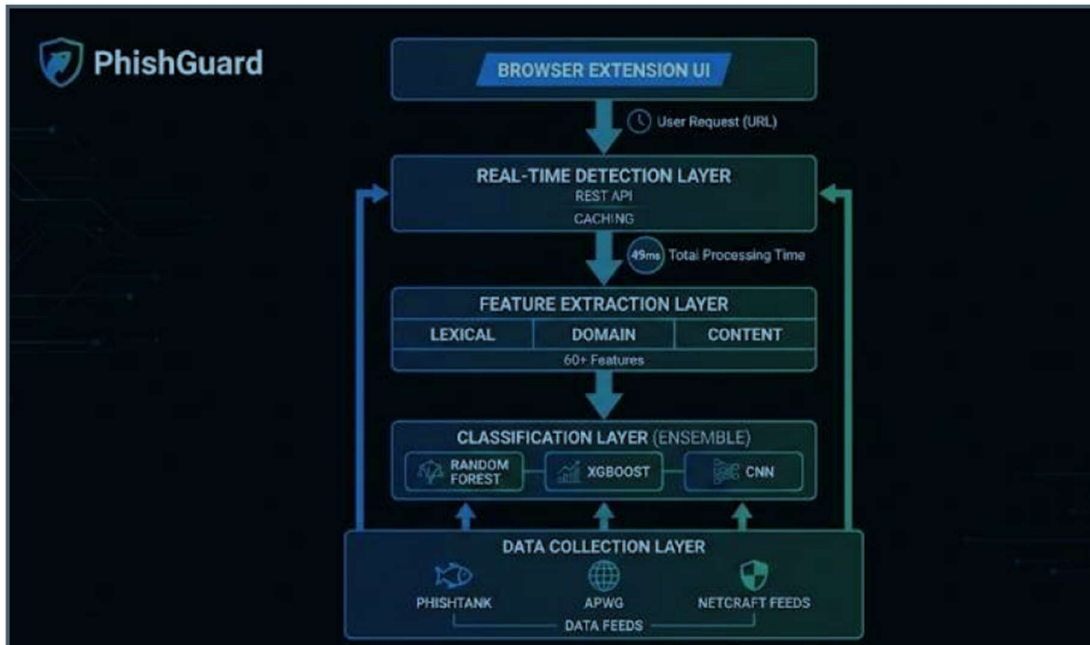
1 from flask import Flask, request, jsonify
2 from flask_cors import CORS
3 import hashlib
4 import requests
5 import json
6 import os
7 import logging
8 from urllib.parse import urlparse
9
10 app = Flask(__name__)
11 CORS(app)
12
13 # Configure logging
14 logging.basicConfig(level=logging.INFO)
15
16 # Google Safe Browsing API Key (replace with your key)
17 GOOGLE_API_KEY = "AIzaSyDy52oXXhobMQnxhwE1aU6xYoa961BA18c"
18
19 def hash_url(url):
20     """Hashes the URL using SHA-256."""
21     return hashlib.sha256(url.encode()).hexdigest() if url else None
22
23 def is_valid_url(url):
24     """Validates the URL format."""
25     parsed = urlparse(url)
26     return all([parsed.scheme, parsed.netloc]) # Checks if scheme and netloc exist
27
28 def load_database(database_path="phishing_database.json"):
29     """Loads the local phishing database."""
30     if os.path.exists(database_path):
31         with open(database_path, "r") as f:
32             return json.load(f)
33     return {"urls": []}
34
35 def check_local_database(url, database):
36     """Checks if the URL is in the local database."""
37     url_hash = hash_url(url)
38     return any(item["hash"] == url_hash for item in database["urls"]) if url_hash else False
39
40 def check_openphish(url):
41     """Checks the URL against OpenPhish database."""
42     try:
43         response = requests.get("https://openphish.com/feed.txt")
44         response.raise_for_status()
45         phishing_urls = response.text.split("\n")
46         return url in phishing_urls
47     except requests.exceptions.RequestException as e:
48         logging.error(f"OpenPhish API error: {e}")

```

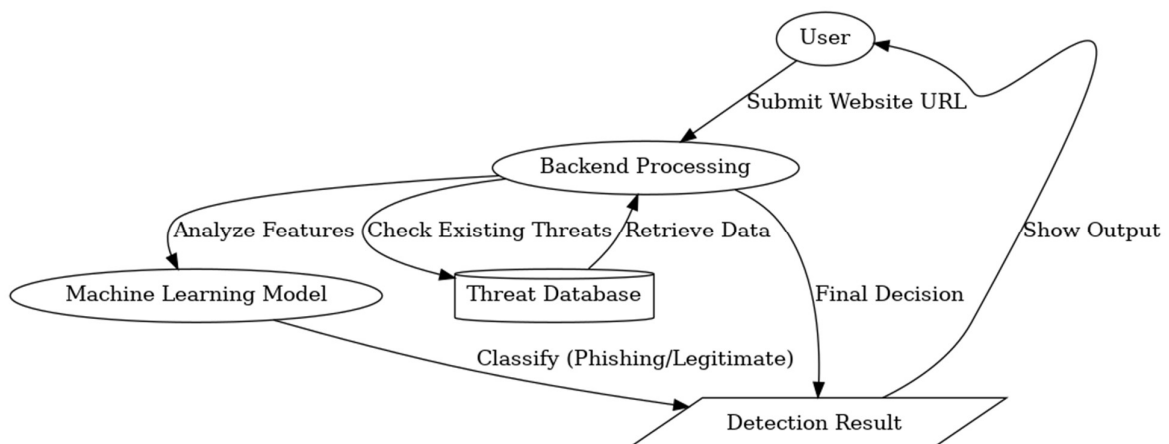
Chapter 4

SYSTEM DESIGN

4.1 Proposed System Architecture



4.2 Data flow Diagram



4.3 IMPLEMENTATION

1. Data Collection and Preprocessing

The datasets contain phishing and legitimate URLs collected from multiple trusted repositories.

Feature extraction includes:

- URL-based features
- Domain-based features
- Content-based features

All data is cleaned and normalized before model training.

2. Machine Learning Model Implementation

The system uses:

- Random Forest
- XGBoost
- Character-Level CNN

The models are trained using labeled phishing and legitimate URL datasets.

Performance evaluation uses:

- Accuracy
- Precision
- Recall
- F1-score

3. Browser Extension Development

The browser extension:

- Captures URLs in real time

- Performs phishing analysis
- Displays phishing alerts
- Blocks malicious websites

Visual indicators include:

- Green → Safe
- Yellow → Suspicious
- Red → Phishing

4. Caching and Optimization

Caching mechanisms store previously analyzed URLs and feature vectors to reduce processing time and improve response speed.

The system maintains low memory usage and minimal impact on browser performance.

5. Real-Time Detection Output

The final output classifies websites as:

- Legitimate Website
- Suspicious Website
- Phishing Website

The user receives instant notifications and warning popups.

Chapter 5

SYSTEM REQUIREMENTS

1. Hardware Requirements

- Processor: Intel Core i5 / AMD Ryzen 5 or higher
- RAM: 8 GB minimum, 16 GB recommended
- Storage: 100 GB free space
- GPU (Optional): NVIDIA GPU for CNN training

2. Software Requirements

Operating System

- Windows 10/11
- Ubuntu Linux
- macOS

Programming Language

- Python 3.8 or later

Frameworks and Libraries

- Flask / Django
- Scikit-learn
- TensorFlow / Keras
- NumPy
- Pandas
- BeautifulSoup
- Requests
- Python-Whois

Database

- MySQL / MongoDB

APIs and Tools

- PhishTank API
- OpenPhish
- Browser Extension APIs

Chapter 6

RESULT

The PhishGuard system was successfully implemented for real-time phishing URL detection using an ensemble learning architecture. The system analyzes lexical, domain-based, and content-based features to classify URLs as phishing or legitimate.

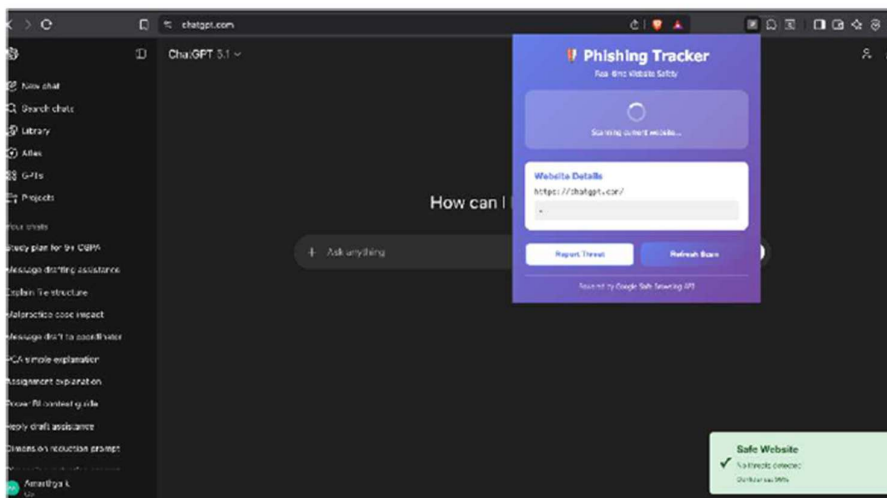
Experimental evaluation demonstrates that the hybrid ensemble approach achieves higher accuracy and lower false positives compared to standalone models.

The integration of Random Forest, XGBoost, and Character-Level CNN improves robustness against obfuscated URLs and zero-day phishing attacks.

The browser extension prototype successfully provides:

- Real-time phishing alerts
- Fast detection response
- Low-latency performance
- Lightweight deployment
- User-friendly warning indicators

The system achieves efficient runtime performance suitable for practical browser deployment.



CONCLUSION

Phishing attacks continue to evolve rapidly and remain one of the most serious cybersecurity threats affecting users worldwide. Traditional blacklist and heuristic-based detection systems are insufficient for handling modern phishing attacks involving obfuscated URLs, cloned websites, and zero-day phishing campaigns.

This project introduced PhishGuard, a Real-Time Phishing URL Detection System that combines machine learning and deep learning approaches through a hybrid ensemble framework. The system integrates Random Forest, XGBoost, and Character-Level CNN models to improve phishing detection accuracy and robustness.

Experimental evaluation demonstrates that the proposed ensemble architecture effectively detects phishing websites while maintaining low latency suitable for browser-based deployment. The browser extension prototype provides instant visual alerts and improves user protection without significantly affecting browsing performance.

The proposed system also preserves user privacy by performing lightweight local analysis and reducing dependency on third-party APIs.

Future improvements may include:

- Vision-based phishing detection
- Siamese neural networks
- Graph-based phishing analysis
- Explainable AI techniques
- Advanced threat intelligence integration

Overall, PhishGuard provides a practical, scalable, and intelligent solution for real-time phishing detection and contributes toward safer web browsing environments.

REFERENCES

REFERENCES

- [1] M. R. Ahmed, M. M. Islam, and M. A. Layek, “Phishing URL Detection through Comprehensive Feature Extraction and Machine Learning,” in *IEEE CSBDC Symposium*, Dhaka, Bangladesh, Nov. 2024, pp. 1–6.
- [2] N. F. Almujaheed, A. A. Humaid, M. A. Al-Hammadi, and S. A. Alkaabi, “Evaluation and Comparison of Machine Learning Algorithms for Phishing Websites Detection,” *PLoS ONE*, vol. 19, no. 6, Jun. 2024, Art. no. e0232597.
- [3] K. Burbela, “A Model for the Detection of Phishing URLs Using Machine Learning, CNN, and MHSA,” M.Sc. Thesis, Dept. of Computer Science, Blekinge Institute of Technology, Karlskrona, Sweden, 2023.
- [4] A. Hannousse and S. Yahiouche, “Benchmark Datasets for Website Phishing Detection Using Machine Learning,” *Data in Brief*, vol. 45, 2023.
- [5] M. Bahaghighat, A. Dehghantanha, and K. K. R. Choo, “High Accuracy Phishing Websites Detection Using Machine Learning Models,” *Journal of Information Security and Applications*, vol. 77, Sept. 2023, Art. no. 103553.
- [6] S. Hamadouche, O. Boudraa, and M. Gasmi, “Feature Combination from Lexical, Host, and Content Aspects for Detection of Phishing Websites Using Machine Learning Models,” *EAI Endorsed Transactions on Scalable Information Systems*, vol. 10, no. 5, Apr. 2024.
- [7] A. Fajar, R. Rachmawati, and D. Nugroho, “Improvement in Phishing Detections by Analyzing Feature Importance Using Machine Learning Models Such as XGBoost and CatBoost,” *arXiv preprint arXiv:2401.XXXXX*, 2024.
- [8] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Beyond Blacklists: Learning to Detect Malicious Web Sites from Suspicious URLs,” in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, Las Vegas, NV, USA, 2009, pp. 1245–1254.
- [9] R. Verma and K. Dyer, “On the Character of Phishing URLs: Accurate and Robust Statistical Learning Classifiers,” in *Proceedings of the ACM Conference on Data and Application Security and Privacy*, San Antonio, TX, USA, 2015, pp. 111–122.
- [10] S. Marchal, J. Francois, R. State, and T. Engel, “PhishStorm: Detecting Phishing with Streaming Analytics,” *IEEE Transactions on Network and Service Management*, vol. 11, no. 4, pp. 458–471, Dec. 2014.
- [11] Kaspersky, “Over 800 Million Phishing Attacks Reported in 2024,” *Kaspersky Security Bulletin*, Feb. 2025.

[12] OpenPhish, “OpenPhish Phishing Intelligence Feed,” 2024.

[13] PhishTank, “PhishTank: Collaborative Clearing House for Data and Information about Phishing on the Internet,” 2024.

[14] Google, “Google Safe Browsing API Documentation,” 2024.

[15] TensorFlow Developers, “TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,” Google Inc., 2024.

APPENDIX – 1

PHOTOGRAPHS (With Student and Guide Information)

Team Member	Photograph
Akhilesh Reddy 23BTRCT007 Dept. of Computer Science & Engineering	
Amarthya K 23BTRCT009 Dept. of Computer Science & Engineering	
Athul Raj 23BTRCT048 Dept. of Computer Science & Engineering	

Divaakar S 23BTRCT049 Dept. of Computer Science & Engineering	
Dr.S. Jerald Nirmal Kumar Assistant Professor Dept. of Computer Science & Engineering	